

HW1: (5 倍分數)

You are an engineer who is responsible for designing a Windows-like GUI systems. You are developing the SDK for application programmers to develop applications under your GUI system.

The following Java example illustrates a base window class SimpleWindow. Soon, you find out that many other GUI components, such as vertical scroll bar, horizontal scroll bar, close button, min/max button (e.g., ) , etc. should be added to a SimpleWindow by programmer's *free will*.

```
// the Window interface class
public interface Window {
    public void draw(); // draws the Window
    public String getDescription(); // returns a description of the Window
}

// extension of a simple Window without any scrollbars
class SimpleWindow implements Window {
    public void draw() {
        // draw window
    }

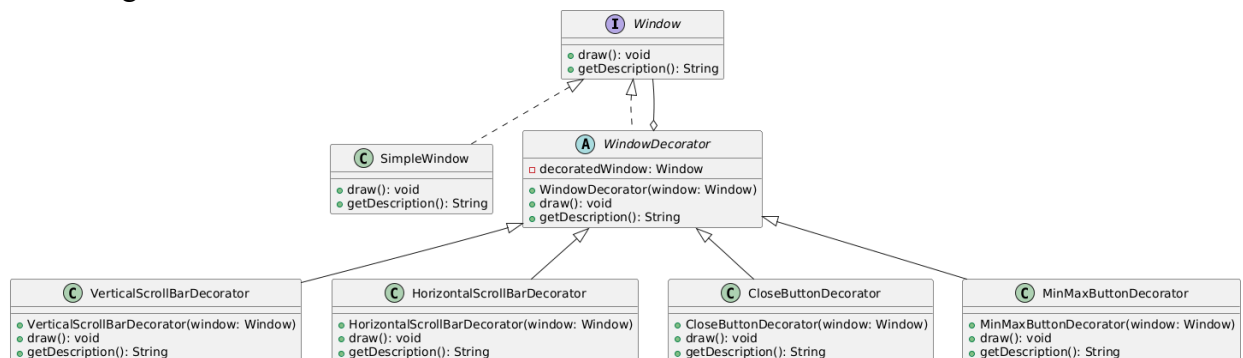
    public String getDescription() {
        return "simple window";
    }
}
```

Your boss knows you are a design pattern expert because you should be well-trained from NCU CSIE. Please propose a good solution to your boss by:

- Draw UML diagrams
- Explain your solution
- Show sample code to demonstrate how application programmer can use your design to new an *SimpleWindow*, add a *verticalscrollbar*, and then add a *closebutton*.

A:

1. UML diagrams:



2. Solution:

這個設計的核心想法是：所有視窗相關物件都實作同一個 Window interface，因此不管是原本的 SimpleWindow，還是加上 scrollbar、button 之後的視窗，在使用者眼中都還是一個 Window。

SimpleWindow 是最基本的視窗，只負責畫出基本視窗內容。

WindowDecorator 是抽象裝飾類別，裡面會保存一個 Window 物件。它本身也實作 Window interface，所以 Decorator 可以包住 SimpleWindow，也可以包住另一個 Decorator。

例如：

```
new CloseButtonDecorator(new VerticalScrollBarDecorator(new SimpleWindow()));
```

3. Sample Java code:

```
// Window interface
```

```
public interface Window {  
    void draw();  
    String getDescription();  
}
```

```
// Basic SimpleWindow
```

```
public class SimpleWindow implements Window {
```

```
    @Override
```

```
    public void draw() {  
        System.out.println("Draw simple window");  
    }
```

```
    @Override
```

```
    public String getDescription() {  
        return "Simple Window";  
    }
```

```
}
```

```
// Abstract Decorator
```

```
public abstract class WindowDecorator implements Window {
```

```
    protected Window decoratedWindow;
```

```
    public WindowDecorator(Window decoratedWindow) {  
        this.decoratedWindow = decoratedWindow;  
    }
```

```
    @Override
```

```
    public void draw() {  
        decoratedWindow.draw();  
    }
```

```

@Override
public String getDescription() {
    return decoratedWindow.getDescription();
}
}

// Vertical Scroll Bar Decorator
public class VerticalScrollBarDecorator extends WindowDecorator {

    public VerticalScrollBarDecorator(Window decoratedWindow) {
        super(decoratedWindow);
    }

    @Override
    public void draw() {
        super.draw();
        drawVerticalScrollBar();
    }

    private void drawVerticalScrollBar() {
        System.out.println("Draw vertical scroll bar");
    }

    @Override
    public String getDescription() {
        return super.getDescription() + ", with vertical scroll bar";
    }
}

// Close Button Decorator
public class CloseButtonDecorator extends WindowDecorator {

    public CloseButtonDecorator(Window decoratedWindow) {
        super(decoratedWindow);
    }

    @Override
    public void draw() {
        super.draw();
        drawCloseButton();
    }

    private void drawCloseButton() {

```

```

        System.out.println("Draw close button");
    }

    @Override
    public String getDescription() {
        return super.getDescription() + ", with close button";
    }
}

// Application Programmer Usage Example
public class Main {
    public static void main(String[] args) {

        Window window = new SimpleWindow();

        window = new VerticalScrollBarDecorator(window);

        window = new CloseButtonDecorator(window);

        window.draw();

        System.out.println(window.getDescription());
    }
}

```

HW2: (3 倍分數)

想像一下，如果你”硬是”要將你的 UML editor 改成 MVC。請問一下你的 model 應該長成什麼樣子？請寫個 class template, 幾個代表性的 API 就好。

A:

如果硬是要把 UML editor 改成 MVC，Model 不應該處理畫面，也不應該直接處理滑鼠事件。

Model 應該負責：

- 儲存 UML 圖的資料
- 儲存所有 UML shapes
- 儲存 class diagram 的 class、attribute、method
- 儲存 object relationship，例如 association、aggregation、composition、generalization
- 提供新增、刪除、修改、查詢 API
- 通知 View 資料改變了

1. Model Class Template:

```
import java.util.ArrayList;
import java.util.List;

public class UMLDiagramModel {

    private List<UMLShape> shapes;
    private List<UMLConnection> connections;
    private List<ModelListener> listeners;

    public UMLDiagramModel() {
        this.shapes = new ArrayList<>();
        this.connections = new ArrayList<>();
        this.listeners = new ArrayList<>();
    }

    // Add a UML shape into model
    public void addShape(UMLShape shape) {
        shapes.add(shape);
        notifyModelChanged();
    }

    // Remove a UML shape from model
    public void removeShape(UMLShape shape) {
        shapes.remove(shape);
        notifyModelChanged();
    }
}
```

```

// Add a relationship between two UML shapes
public void addConnection(UMLConnection connection) {
    connections.add(connection);
    notifyModelChanged();
}

// Remove a relationship
public void removeConnection(UMLConnection connection) {
    connections.remove(connection);
    notifyModelChanged();
}

// Move a shape
public void moveShape(String shapeId, int newX, int newY) {
    UMLShape shape = findShapeById(shapeId);
    if (shape != null) {
        shape.setPosition(newX, newY);
        notifyModelChanged();
    }
}

// Rename a class shape
public void renameClass(String shapeId, String newName) {
    UMLShape shape = findShapeById(shapeId);

    if (shape instanceof UMLClassShape) {
        ((UMLClassShape) shape).setClassName(newName);
        notifyModelChanged();
    }
}

// Add attribute to class
public void addAttribute(String shapeId, String attribute) {
    UMLShape shape = findShapeById(shapeId);

    if (shape instanceof UMLClassShape) {
        ((UMLClassShape) shape).addAttribute(attribute);
        notifyModelChanged();
    }
}

// Add method to class

```

```

public void addMethod(String shapeId, String method) {
    UMLShape shape = findShapeById(shapeId);

    if (shape instanceof UMLClassShape) {
        ((UMLClassShape) shape).addMethod(method);
        notifyModelChanged();
    }
}

// Get all shapes
public List<UMLShape> getShapes() {
    return shapes;
}

// Get all connections
public List<UMLConnection> getConnections() {
    return connections;
}

// Find shape by id
public UMLShape findShapeById(String shapeId) {
    for (UMLShape shape : shapes) {
        if (shape.getId().equals(shapeId)) {
            return shape;
        }
    }
    return null;
}

// Observer pattern for MVC
public void addModelListener(ModelListener listener) {
    listeners.add(listener);
}

public void removeModelListener(ModelListener listener) {
    listeners.remove(listener);
}

private void notifyModelChanged() {
    for (ModelListener listener : listeners) {
        listener.modelChanged();
    }
}

```

```
}
```

2. UMLShape Base Class

```
public abstract class UMLShape {  
  
    protected String id;  
    protected int x;  
    protected int y;  
    protected int width;  
    protected int height;  
  
    public UMLShape(String id, int x, int y, int width, int height) {  
        this.id = id;  
        this.x = x;  
        this.y = y;  
        this.width = width;  
        this.height = height;  
    }  
  
    public String getId() {  
        return id;  
    }  
  
    public void setPosition(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    public boolean containsPoint(int mouseX, int mouseY) {  
        return mouseX >= x && mouseX <= x + width  
            && mouseY >= y && mouseY <= y + height;  
    }  
}
```

當 Controller 接收到使用者操作，例如新增類別、移動圖形、建立連線時，Controller 不會直接修改畫面，而是呼叫 Model 的 API，例如：

```
model.addShape(new UMLClassShape("C1", "Student", 100, 100));  
model.addAttribute("C1", "- name: String");  
model.addMethod("C1", "+ getName(): String");  
model.moveShape("C1", 200, 150);
```

Model 更新資料後，透過 notifyModelChanged() 通知 View 重新繪製畫面。這樣可以讓資料邏輯、畫面顯示、使用者操作分開，符合 MVC 的精神。